

Self-Hosted Teams-like Meeting Platform — Design Document

1. Project Overview

Goal

Build a fully self-hosted Teams-like video meeting platform with:

- Group video calling
- Audio calling
- Screen sharing
- Real-time chat
- Meeting recording
- Local file storage
- Zero third-party dependency

The platform must run on:

- Local machine
- Self-hosted VPS
- Docker environment

No dependency on:

- LiveKit Cloud
- Agora
- Twilio
- Zoom SDK
- Daily.co

2. MVP Objectives

Initial Goals

Phase 1

- Multi-user video meeting
- Room creation
- Join room
- Audio/video controls
- Screen sharing

Phase 2

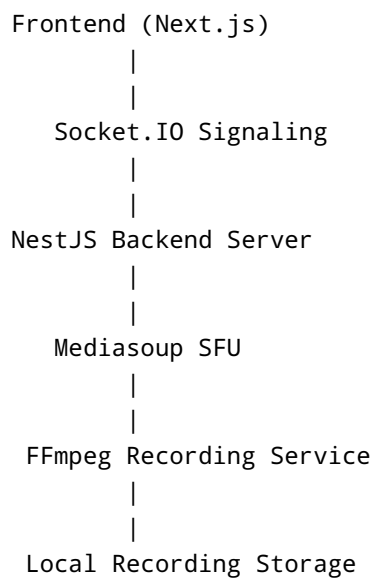
- Meeting recording
- Save recordings locally
- Chat system

Phase 3

- Meeting history
- File uploads
- Authentication

3. System Architecture

High-Level Architecture



4. Technology Stack

Frontend Stack

Technology	Purpose
Next.js	Frontend framework
React	UI rendering

Technology	Purpose
TypeScript	Type safety
Tailwind CSS	Styling
Socket.IO Client	Realtime signaling
mediasoup-client	WebRTC client

Backend Stack

Technology	Purpose
NestJS	Backend framework
Socket.IO	Signaling server
mediasoup	SFU media server
FFmpeg	Recording engine
SQLite	Local database
Docker	Containerization

5. Why Mediasoup

Benefits

Fully Self-Hosted

All media infrastructure runs on your own machine.

Zero Vendor Lock-in

No dependency on external providers.

SFU Architecture

Better performance than mesh WebRTC.

Scalability

Supports larger meetings efficiently.

Production Ready

Used in large-scale real-time communication systems.

6. SFU Architecture Design

Mesh vs SFU

Mesh Architecture (Bad for Scaling)

A ↔ B ↔ C ↔ D

Problems:

- High CPU usage
 - High bandwidth
 - Browser overload
 - Poor scalability
-

SFU Architecture (Recommended)

Users → Mediasoup SFU → Other Users

Benefits:

- Low bandwidth usage
 - Better scalability
 - Efficient media routing
 - Lower CPU consumption
-

7. Frontend Architecture

Frontend Responsibilities

Meeting UI

- Video grid
- Active speaker view
- Chat panel

- Participant list
- Screen sharing controls

Realtime Features

- Room joining
- Media transport setup
- Message updates
- Connection state

WebRTC Features

- Camera access
 - Microphone access
 - Stream publishing
 - Stream consuming
-

8. Frontend Folder Structure

```
frontend/  
├─ app/  
├─ components/  
├─ hooks/  
├─ services/  
├─ store/  
├─ socket/  
├─ webrtc/  
├─ styles/  
└─ utils/
```

9. Backend Architecture

Backend Responsibilities

Authentication

- Temporary session handling
- User management

Room Management

- Create rooms

- Join rooms
- Leave rooms

Signaling

- ICE candidate exchange
- DTLS negotiation
- Transport creation

Recording

- Start recording
- Stop recording
- Manage FFmpeg workers

10. Backend Modules

```
backend/  
├── src/  
│   ├── auth/  
│   ├── rooms/  
│   ├── signaling/  
│   ├── mediasoup/  
│   ├── recording/  
│   ├── websocket/  
│   ├── database/  
│   └── common/
```

11. Mediasoup Design

Core Concepts

Worker

Handles media processing.

Router

Routes media streams.

Transport

WebRTC transport connection.

Producer

Publishes audio/video stream.

Consumer

Receives stream.

12. Meeting Flow

Meeting Creation Flow

```
graph TD; A[User Creates Room] --> B[Backend Generates Room ID]; B --> C[Mediasoup Router Created]; C --> D[Participants Join]; D --> E[WebRTC Transport Established]; E --> F[Media Streaming Starts];
```

13. WebRTC Signaling Flow

```
graph TD; A[Client] --> B[Socket.IO Event]; B --> C[NestJS Signaling Server]; C --> D[Mediasoup Router]; D --> E[Transport Information];
```

14. Socket.IO Events

Room Events

- create-room
- join-room
- leave-room

Media Events

- create-transport
- connect-transport
- produce
- consume

Meeting Events

- mute-user
- camera-toggle
- start-screen-share

Recording Events

- recording-start
- recording-stop

Chat Events

- send-message
 - receive-message
-

15. Recording Architecture

Recording Goal

Record complete meetings and save locally.

Recording Flow

```
Meeting Starts
  ↓
Host Starts Recording
  ↓
Mediasoup Creates RTP Stream
  ↓
FFmpeg Receives RTP Stream
  ↓
Video Encoded as MP4
  ↓
Saved to recordings folder
```

16. FFmpeg Integration

FFmpeg Responsibilities

Recording

- Capture RTP streams
- Encode media
- Generate MP4

Encoding

- H264 video
- AAC audio

File Management

- Naming
- Storage
- Cleanup

17. Recording Storage Design

Local Storage Folder

```
/recordings
```

Example Files

```
meeting-001.mp4
meeting-002.mp4
meeting-003.mp4
```

18. Database Design

Database Choice

SQLite

Reason:

- Zero cost
- Easy setup
- Lightweight
- Perfect for MVP

19. Database Tables

Users Table

```
users
- id
- username
- created_at
```

Rooms Table

```
rooms
- id
- room_name
- created_by
- created_at
```

Participants Table

```
participants
- id
- room_id
- user_id
- joined_at
```

Recordings Table

```
recordings
- id
- room_id
- file_path
- duration
- created_at
```

20. Local Development Environment

Development Components

Component	Runs Locally
Frontend	Yes
Backend	Yes
Mediasoup	Yes
SQLite	Yes
FFmpeg	Yes
Recording Storage	Yes

21. Docker Architecture

Containers

```
Docker Compose
├── frontend
├── backend
├── mediasoup
└── redis
```

22. Security Design

Basic Security

Authentication

- JWT tokens
- Temporary guest sessions

Media Security

- DTLS encryption
- SRTP encryption

API Protection

- Input validation
- Rate limiting

23. Frontend Pages

Landing Page

- Create meeting
- Join meeting

Lobby Page

- Camera preview
- Mic preview

- Username input

Meeting Room

- Video grid
 - Controls
 - Chat
 - Recording controls
-

24. Meeting Controls

Controls Included

- Mic mute/unmute
 - Camera on/off
 - Screen sharing
 - Leave meeting
 - Start recording
 - Stop recording
-

25. Chat System

Chat Features

- Real-time messaging
 - Participant messages
 - Meeting chat panel
-

26. Screen Sharing Design

Browser API

Use:

- `getDisplayMedia()`

Flow

```
graph TD; A[User Shares Screen] --> B[Browser Captures Stream]; B --> C[Mediasoup Publishes Stream]; C --> D[Participants Receive Stream];
```

27. Media Handling Design

Audio Codec

- Opus

Video Codec

- VP8 initially
- H264 later

28. Performance Optimization

Initial Optimization

- Limit video quality
- Pause unused consumers
- Use simulcast later

29. Scalability Plan

MVP Capacity

Recommended

- 5–10 concurrent users
- Single VPS
- Single Mediasoup worker

Future Scaling

Upgrade Path

- PostgreSQL
 - Redis Pub/Sub
 - Multiple Mediasoup workers
 - Kubernetes
 - Cloud storage
-

30. Hardware Requirements

Local Development

Resource	Minimum
CPU	4 cores
RAM	8 GB
Storage	50 GB SSD

31. Production VPS Recommendation

Recommended VPS

Resource	Recommended
CPU	4-8 cores
RAM	8-16 GB
Storage	100 GB SSD
Bandwidth	High-speed

32. Development Roadmap

Week 1

Goals

- Setup Next.js
 - Setup NestJS
 - Setup Socket.IO
 - Create room system
-

Week 2

Goals

- Integrate Mediasoup
 - WebRTC transports
 - Multi-user video calls
-

Week 3

Goals

- Chat system
 - Screen sharing
 - Meeting controls
-

Week 4

Goals

- FFmpeg recording
 - Save recordings locally
 - Meeting history
-

33. Main Technical Challenges

WebRTC Complexity

- ICE negotiation
- NAT traversal
- Browser compatibility

Recording Complexity

- RTP handling
- Media synchronization
- CPU-intensive encoding

Scaling Challenges

- Multiple meetings
 - Concurrent streams
 - Bandwidth management
-

34. Future Features

Future Upgrades

- Authentication system
 - Cloud storage
 - AI summaries
 - Mobile applications
 - Webinar mode
 - Whiteboard
 - Noise suppression
 - Background blur
-

35. Final Recommended Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

- NestJS
- Socket.IO

Media Server

- Mediasoup

Recording

- FFmpeg

Database

- SQLite

Infrastructure

- Docker Compose

Hosting

- Local machine initially

36. Final Summary

This architecture provides:

- Fully self-hosted infrastructure
- Zero third-party meeting dependency
- Local meeting recording
- Group video calls
- Real-time communication
- Future scalability

The system is designed as a strong MVP foundation for building a custom Teams-like communication platform with complete ownership of media, storage, and infrastructure.